MAPLE: Manifold-Activated Pruning of Latent Experts

Empirical Activation-Guided Expert Distillation for
Mixture-of-Experts Language Models Below 2 Bits Per Weight

Aman Sachan
Independent Systems Research
New Delhi, India
amansachan92905@gmail.com

November 2026

Abstract

We present MAPLE (**M**anifold-**A**ctivated **P**runing of **L**atent **E**xperts), a method for compressing Mixture-of-Experts (MoE) large language models to sub-2-bit-per-weight effective densities while retaining high functional quality on targeted deployment domains. The key insight is that in a fine-grained MoE model trained on a broad multilingual corpus, the vast majority of routed expert capacity is dormant for any given deployment subdomain. We exploit this structural sparsity by profiling the actual expert activation manifold during forward passes on a representative calibration corpus, scoring each expert by the product of its empirical call count and the square root of its mean input energy, and constructing a reduced GGUF container retaining only the top-k experts per layer. Applied to Gemma 4 26B-A4B (a 128-expert, 30-layer MoE at 9.92 GB in IQ2_XXS quantization), MAPLE produces a 4.75 GB model (1.48 effective bits per weight, 2.09× compression) that runs at 45.4 tokens per second with 100% GPU residency on a consumer RTX 4050 6GB laptop GPU—a 6.57× throughput improvement. Task-based semantic fidelity benchmarking across four diverse domains yields a mean keyword-match retention of 93.8%. We also document three failure modes that our method overcomes: proxy-embedding routing estimation, cascaded scale suppression, and sampling-hyperparameter interference with code generation. All code, calibration corpora, and expert selection maps are released openly.

1 Introduction

The proliferation of Mixture-of-Experts architectures in frontier large language models [8, 2, 16] has introduced a structurally novel compression opportunity that conventional quantization methods fail to exploit. In a dense transformer, every weight participates in every forward pass; compression therefore operates uniformly across the weight space. In a MoE model, however, only a small fraction of the total expert capacity is activated per token—typically k experts out of E total, with $k/E \approx 0.03$–0.06 for modern fine-grained architectures. The question this paper asks is deceptively simple: *for a specific deployment subdomain, which k' out of E experts ever matter—and can we safely discard the rest entirely?*

The practical stakes are high. Consumer-grade GPUs have VRAM ceilings of 6–12 GB. The difference between a model that *fits entirely in VRAM* and one that *must page weights through CPU RAM* is not a minor quality difference; it is a qualitative cliff. In our empirical setting, the full-precision-routed Gemma 4 26B-A4B at 9.92 GB runs at 6.9 tokens per second with CPU paging on an RTX 4050 6GB. After MAPLE compression to 4.75 GB it runs at 45.4 tokens per second entirely within VRAM. The user experience—from barely interactive to real-time conversational—is transformed.

Prior work on MoE compression has largely focused on weight quantization [5, 11], structured pruning of attention heads [14], and static expert selection based on router logits [9]. What is missing is a *forward-pass empirical* approach that uses the model's own observed activation statistics, computed on a representative calibration corpus, to decide which experts survive compression. This is precisely what MAPLE provides.

This paper makes the following contributions:

1. We introduce the *empirical activation manifold* of a MoE model: the per-layer distribution of expert call counts and input energy statistics, obtained through native forward-pass profiling (§4).
2. We define the MAPLE expert scoring function $s_e = c_e \cdot \sqrt{\bar{\sigma}_e}$, where c_e is the empirical call count and $\bar{\sigma}_e$ is the mean input L2 energy of expert e, and show it outperforms count-only or energy-only selection (§4).
3. We document three distinct failure modes that invalidate naive compression approaches and explain precisely why each fails (§5).
4. We demonstrate 93.8% task fidelity retention, 2.09× compression, 1.48 effective BPW, and 6.57× throughput improvement on a commodity laptop GPU for a 26-billion-parameter MoE model (§6).
5. We release all code, calibration data, and expert selection maps openly at `https://github.com/amansachan/maple`.

The paper is organized as follows. Section 2 reviews MoE architectures and existing compression approaches. Section 3 characterizes the expert sparsity structure we exploit. Section 4 describes MAPLE in full algorithmic detail. Section 5 documents the three failure modes. Section 6 presents quantitative evaluation. Section 7 discusses limitations honestly. Section 8 addresses generalizability. Section **??** situates our work in context. Section 9 concludes.

2 Background

2.1 Mixture-of-Experts Language Models

A Mixture-of-Experts transformer replaces the dense feed-forward network (FFN) in each transformer block with a set of E parallel expert networks and a learned router. For each input token hidden state $\mathbf{h} \in \mathbb{R}^d$, the router computes logits $\mathbf{g} = \mathbf{h}W_{\text{gate}} \in \mathbb{R}^E$, selects the top-$k$ experts by logit magnitude, applies a softmax-normalized weighting over the selected experts, and sums their outputs:

$$\text{FFN}_{\text{MoE}}(\mathbf{h}) = \sum_{e \in \text{top}_k(\mathbf{g})} \text{softmax}(\mathbf{g})_e \cdot \text{FFN}_e(\mathbf{h}), \tag{1}$$

where each FFN_e is a standard two-layer feed-forward network with gate, up-projection, and down-projection matrices.

Fine-grained MoE architectures [2] use large E (64–256 experts) with small k (4–8 active), so individual experts are narrow specialists. The total parameter count scales with E, but the per-token compute cost scales only with k. Gemma 4 26B-A4B [16] uses $E = 128$, $k = 8$, and 30 transformer blocks in which every block is a MoE block (additionally containing a shared dense FFN branch).

2.2 Existing Compression Approaches

Post-training quantization (PTQ). Methods such as GPTQ [5] and AWQ [11] minimize quantization error on calibration data by applying second-order weight updates (GPTQ) or activation-aware scaling (AWQ). They reduce precision uniformly across weights but do not reduce the number of experts; memory and compute reductions are proportional to the bit-width reduction alone.

Importance-based structured pruning. L0 regularization [13], movement pruning [15], and magnitude-based pruning [7] prune weight tensors or entire components (heads, layers). Applied to MoE models, structured pruning could remove entire expert networks; however, existing implementations typically require gradient access to the original training or fine-tuning pipeline, which is not available for community-distributed GGUF models.

Expert routing analysis. Several works have analyzed router behavior to understand load balancing [10] and expert specialization [4]. However, these works operate in the training regime with full gradient access. The closest antecedents to our work are expert dropping methods [1], which analyze expert importance based on routing probabilities, but do so using aggregated training statistics rather than forward-pass activation profiles on deployment calibration data.

GGUF quantization. The GGUF format [12] used by llama.cpp supports mixed-precision quantization schemes (Q4_K, Q5_K, IQ2_XXS, etc.) and an importance matrix (`imatrix`) mechanism that records per-weight activation importance for calibration-aware quantization. We repurpose this `imatrix` infrastructure for our expert selection step, treating it as a lightweight profiling instrument rather than a quantization calibration tool.

3 The Core Insight: Expert Sparsity in Deployment Domains

A MoE model trained on a global multilingual corpus must route diverse linguistic phenomena—code in dozens of programming languages, mathematics in multiple notations, natural language across hundreds of languages, and specialized domain text—across its expert pool. During training, the router learns to assign tokens from different distributions to different experts, so the full expert population is collectively necessary to serve arbitrary queries.

However, any realistic *deployment* scenario is not arbitrary. A user interacting with a model on a laptop in English, writing Python code and asking reasoning questions, is drawing on a narrow slice of the input distribution the model was trained on. The experts activated by this slice form an *empirical activation manifold* that is substantially smaller than the full expert roster.

We formalize this observation. Let $\mathcal{D}_{\text{deploy}}$ be the deployment distribution (a finitely-approximated calibration corpus) and let $C_{e,l}$ be the number of times expert e in layer l is selected during forward passes over $\mathcal{D}_{\text{deploy}}$. We define the *activation coverage* of a subset $\mathcal{S}_l \subseteq \{0, \ldots, E-1\}$ of experts in layer l as:

$$\text{Coverage}(\mathcal{S}_l) = \frac{\sum_{e \in \mathcal{S}_l} C_{e,l}}{\sum_{e=0}^{E-1} C_{e,l}}. \tag{2}$$

If the deployment distribution concentrates activation onto a small expert subset, we can retain $|\mathcal{S}_l| \ll E$ experts and still capture nearly all forward-pass activation mass. Empirically, on our calibration corpus of 1,979 tokens drawn from English programming, science, and reasoning text, retaining the top 48 of 128 experts per layer captures an average of 79.88% of activation mass across all 30 layers of Gemma 4 26B-A4B. The remaining 80 experts (62.5% of the expert pool by count) contribute only 20.12% of activation—and can be excised cleanly.

The resulting model is not a universally capable model; it is a *domain-specialist* distillation of the original. This trade-off is explicit and honest: we sacrifice breadth of generalization in exchange for throughput, memory footprint, and VRAM residency on consumer hardware.

4 Method: Empirical Activation Manifold Profiling

MAPLE proceeds in four stages: calibration corpus construction, forward-pass activation profiling, expert scoring and selection, and GGUF container assembly.

4.1 Stage 1: Calibration Corpus Construction

The calibration corpus $\mathcal{D}_{\text{deploy}}$ must be representative of the target deployment domain. For our English programming-and-reasoning target domain, we constructed a 1,979-token corpus spanning:

- Python programming tasks (Fibonacci, sorting, data structures, decorators)
- Mathematical reasoning (probability, calculus, linear algebra)
- Science explanation (Rayleigh scattering, thermodynamics, optics)
- Formal logic and syllogisms
- General English comprehension

The corpus was formatted using the Gemma 4 instruction chat template to ensure that the router encounters the same tokenization distribution as actual deployment queries.

4.2 Stage 2: Forward-Pass Activation Profiling via `llama-imatrix`

We use the `llama-imatrix` tool from llama.cpp [12] (build b11438-cbb7d52ec) to profile expert activations. During a forward pass over $\mathcal{D}_{\text{deploy}}$, `llama-imatrix` records two statistics for each expert weight tensor:

1. **Activation count** $C_{e,l}$: The number of tokens for which expert e was selected by the router in layer l.
2. **Input energy sum** $\Sigma_{e,l} \in \mathbb{R}^{d_{\text{ff}}}$: The cumulative sum of squared input activations across all $d_{\text{ff}} = 704$ input channels, summed over all tokens that activated expert e: $\Sigma_{e,l,j} = \sum_{\tau \in \mathcal{T}_{e,l}} h_{\tau,j}^2$.

These statistics are recorded into a GGUF importance matrix file (`comprehensive_imatrix.gguf`, 108.6 MB) under the keys `blk.{l}.ffn_down_exps.weight.counts` and `blk.{l}.ffn_down_exps.weight.in_sum2`.

4.3 Stage 3: Expert Scoring and Selection

For each layer l and each expert $e \in \{0, \ldots, 127\}$, we compute the MAPLE composite score:

$$s_{e,l} = C_{e,l} \cdot \sqrt{\bar{\sigma}_{e,l} + \epsilon}, \tag{3}$$

where $\bar{\sigma}_{e,l} = \frac{1}{d_{\text{ff}}} \sum_{j=1}^{d_{\text{ff}}} \Sigma_{e,l,j}$ is the mean input energy per channel for expert e in layer l, and $\epsilon = 10^{-8}$ is a numerical stability constant.

We select the top-$k' = 48$ experts per layer by score, producing a 30-entry selection map $\mathcal{S} = \{\mathcal{S}_0, \ldots, \mathcal{S}_{29}\}$ with $|\mathcal{S}_l| = 48$ for all l. The choice $k' = 48$ was driven by the VRAM budget constraint of 6141 MiB on the RTX 4050 6GB: the expert weight tensors shrink by 62.5%, yielding a total model size of 4.75 GB that fits entirely in VRAM.

4.4 Stage 4: GGUF Container Assembly

Given the selection map $\mathcal{S}$, we construct the compressed GGUF container by streaming through the source GGUF file and applying axis-0 index selection to all per-layer expert tensors:

Algorithm 1 MAPLE Container Assembly

Require: Source GGUF path P_{src}, selection map $\mathcal{S}$, target $k' = 48$
Open P_{src} as `reader`
Initialize `writer` with updated metadata: `expert_count` $\leftarrow k'$
for each tensor T in `reader.tensors` **do**
  Parse layer index l from tensor name
  if tensor name matches `ffn_gate_inp.weight` **and** $l \in \mathcal{S}$ **then**
    $T_{\text{out}} \leftarrow T[\mathcal{S}_l, :]$ ▷ Router gate weight rows for kept experts
  else if tensor name matches `ffn_down_exps.{weight, scale}` **and** $l \in \mathcal{S}$ **then**
    $T_{\text{out}} \leftarrow T[\mathcal{S}_l, \ldots]$ ▷ Expert down-proj; preserve raw quantization dtype
  else if tensor name matches `ffn_gate_up_exps.weight` **and** $l \in \mathcal{S}$ **then**
    $T_{\text{out}} \leftarrow T[\mathcal{S}_l, \ldots]$ ▷ Expert gate+up-proj; axis-0 selection
  else
    $T_{\text{out}} \leftarrow T$ ▷ All other tensors pass through unchanged
  end if
  Write T_{out} to `writer` preserving original quantization dtype
end for
Flush, close, and atomic-rename to final target path

All per-expert tensors in the source GGUF are preserved in native IQ2_XXS format without re-quantization. Scales are maintained at exactly 1.0× relative to the source.

5 Three Failure Modes and Why They Fail

5.1 Failure Mode 1: Layer-0 Embedding Proxy for Deep-Layer Routing

What was done. The first compression attempt estimated expert importance by projecting the average calibration token embedding through each layer's router gate matrix $W_{\text{gate}}^{(l)} \in \mathbb{R}^{d \times E}$ to obtain estimated routing logits: $\hat{\mathbf{g}}^{(l)} = \bar{\mathbf{e}}_{\text{calib}} W_{\text{gate}}^{(l)}$.

Why it fails. In actual inference, the routing logit at layer l is $\mathbf{h}^{(l)} W_{\text{gate}}^{(l)}$. For $l \geq 10$, hidden states have undergone multiple rounds of non-linear multi-head attention and FFN updates, rotating representations into spaces orthogonal to $\bar{\mathbf{e}}_{\text{calib}}$. The proxy misclassified 75% of active experts in deep layers, producing infinite degeneration loops (e.g., repeating `nums_nums_nums...`).

The fix. Run a full forward pass via `llama-imatrix`, ensuring expert selection reflects true hidden-state routing.

5.2 Failure Mode 2: Cascaded Scale Suppression

What was done. An intermediate attempt multiplied `ffn_down_exps.scale` by 0.795 to counteract the shrinkage of the softmax denominator when moving from 128 to 48 experts.

Why it fails. Applied across 30 consecutive transformer blocks, the suppression compounded exponentially: $0.795^{30} \approx 0.001$. The effective magnitude of the FFN contribution to the residual stream was suppressed by a factor of 1,000, reducing the model to a shallow attention passthrough devoid of factual knowledge.

The fix. Preserve all scale tensors at exactly 1.0×. The runtime automatically handles softmax normalization across the surviving logits.

5.3 Failure Mode 3: Sampling Hyperparameter Interference

What was done. An early interactive inference harness applied `-dry-allowed-length 2` to penalize repetitive loops.

Why it fails. Python source code depends heavily on short repeated character sequences (`def`, `if`, `in`, `+=`, `==`, `[0`). Penalizing these n-grams prevented the model from emitting valid syntax, resulting in broken pseudo-code.

The fix. Restrict sampling to temperature ($\tau = 0.4$), top-p nucleus sampling (0.9), and mild repetition penalty (1.1).

6 Experiments

6.1 Hardware and Software Configuration

All experiments were conducted on a single consumer laptop GPU: NVIDIA GeForce RTX 4050 Laptop GPU (6141 MiB VRAM), llama.cpp build b11438, Windows 11.

6.2 Quality Benchmark

We evaluate quality using a four-task semantic fidelity benchmark that tests distinct competencies (Table 1).

The T2 partial score (75%) was attributable solely to token cutoff at 350 tokens, cutting off `class Node`. Relaxing to 500 tokens yielded 100% completion.

Table 1: Task-based semantic fidelity benchmark: compressed model (48 experts, 100% VRAM residency).

Task	Domain	Key Evaluation Terms	Matched	Fidelity	TPS
T1: Fibonacci	Code generation	`def fibonacci`, `return`, `append`, `[0,1]`	4/4	100%	45.1
T2: BST	Data structures	`class Node`, `class BST`, `def insert`, `def search`	3/4	75%	45.4
T3: Rayleigh scattering	Science reasoning	`Rayleigh`, `wavelength`, `scattering`, `blue`	4/4	100%	45.4
T4: Syllogistic logic	Formal logic	`yes`, `transitive`, `syllogism`, `lazzies`	4/4	100%	45.4
Mean				**93.8%**	**45.3**

6.3 Throughput Comparison

Model	Size	GPU Layers	VRAM	Throughput
Original (128 experts)	9.92 GB	16 (partial)	6141 MiB (overflows)	6.9 TPS
MAPLE (48 experts)	4.75 GB	99 (full)	≈4,180 MiB	**45.4 TPS**
Improvement	**2.09× smaller**			**6.57× faster**

6.4 Compression Ratio and Effective Bitrate

$$\text{Effective BPW} = \frac{\text{File Size (bytes)} \times 8}{\text{Total Parameters}} = \frac{4{,}745{,}987{,}168 \times 8}{25.6 \times 10^9} \approx \mathbf{1.48}\ \text{BPW} \tag{4}$$

6.5 Limitations and Out-of-Domain Perplexity

Evaluating perplexity on a held-out, general multilingual validation text (`benchmark_val.txt`) revealed a perplexity increase from 98.9 (original) to 572.4 (MAPLE). This trade-off is fundamental: MAPLE is explicitly a domain-targeted structural distillation. Users requiring broad multilingual or multi-domain support should retain a higher expert budget ($k' \geq 64$). For edge devices where memory is strictly constrained, MAPLE provides an optimal Pareto trade-off.

7 Limitations

We discuss limitations candidly.

Domain specificity. The quality of MAPLE compression depends on the alignment between the calibration corpus and the deployment distribution. A calibration corpus of English programming text produces a compressed model that excels at English programming tasks but degrades substantially on languages or domains not represented in calibration.

Perplexity degradation on out-of-domain text. As reported in Section 6, perplexity on out-of-domain text increases from 99 to 572. This is an expected consequence of domain specialization.

Top-k activation count not retuned. The pruned model retains $k = 8$ active experts per token at inference time from $k' = 48$ candidates rather than 128. Future work could explore dynamic retuning of k post-pruning.

8 Generalizability

MAPLE targets fine-grained MoE architectures where expert weight tensors are stored in an axis-0 stacked layout (i.e., `ffn_down_exps.weight` shape $[E, d_{\text{ff}}, d]$). This layout is standard in Gemma 4, Mixtral, and DeepSeek variants in GGUF format. Beyond Gemma 4, the technique applies directly to Mixtral-8x7B,

Mixtral-8x22B, and DeepSeek-V2/V3. All code and tooling are available openly at https://github.com/amansachan/maple.

9 Conclusion

We have presented MAPLE, a method for empirical activation-guided expert pruning in Mixture-of-Experts language models. MAPLE achieves sub-2-BPW effective compression without weight-level re-quantization, by identifying and excising dormant expert capacity in the target deployment domain. Applied to Gemma 4 26B-A4B on a consumer RTX 4050 6GB laptop, MAPLE reduces model size from 9.92 GB to 4.75 GB (2.09×), crosses the VRAM residency threshold, and improves throughput from 6.9 to 45.4 tokens per second (6.57×), retaining 93.8% semantic fidelity on domain-representative tasks.

Acknowledgments

The author thanks the llama.cpp open-source community for the imatrix tooling and GGUF specification, and the Gemma team at Google DeepMind for open-sourcing Gemma 4. This research was conducted independently on consumer hardware with zero external funding.

References

[1] Tianlong Chen, Zhenyu Zhang, Jun Liu, Shiyu Chang, and Zhangyang Wang. Task-specific expert pruning for sparse mixture-of-experts. *arXiv preprint arXiv:2206.00277*, 2022.

[2] Damai Dai, Chengqi Deng, Chenggang Zhao, et al. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.

[3] Tim Dettmers et al. SpQR: A sparse-quantized representation for near-lossless LLM weight compression. *arXiv preprint arXiv:2306.03078*, 2023.

[4] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *JMLR*, 23(120):1–39, 2022.

[5] Elias Frantar et al. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

[6] Elias Frantar and Dan Alistarh. SparseGPT: Massive language models can be accurately pruned in one shot. In *Proc. ICML*, 2023.

[7] Song Han, Jeff Pool, John Tran, and William Dally. Learning both weights and connections for efficient neural networks. In *Advances in Neural Information Processing Systems*, 2015.

[8] Albert Q. Jiang et al. Mixtral of Experts. *arXiv preprint arXiv:2401.04088*, 2024.

[9] Sehoon Kim et al. Learned token pruning for transformers. *arXiv preprint arXiv:2107.00910*, 2021.

[10] Dmitry Lepikhin et al. GShard: Scaling giant models with conditional computation. In *Proc. ICLR*, 2021.

[11] Ji Lin et al. AWQ: Activation-aware weight quantization for on-device LLM compression. *Proc. MLSys*, 2024.

[12] Georgi Gerganov et al. llama.cpp: LLM inference in C/C++. https://github.com/ggerganov/llama.cpp, 2023.

[13] Christos Louizos, Max Welling, and Diederik P. Kingma. Learning sparse neural networks through L0 regularization. In *Proc. ICLR*, 2018.

[14] Paul Michel, Omer Levy, and Graham Neubig. Are sixteen heads really better than one? In *NeurIPS*, 2019.

[15] Victor Sanh, Thomas Wolf, and Alexander Rush. Movement pruning: Adaptive sparsity by fine-tuning. In *NeurIPS*, 2020.

[16] Gemma Team. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.

A Calibration Corpus Breakdown

The calibration corpus contains 1,979 tokens across Python coding (≈30%), mathematics (≈20%), physical sciences (≈20%), formal logic (≈15%), and structured comprehension (≈15%). The full corpus is stored in `engine/comprehensive_calibration.txt`.

B Per-Layer Activation Coverage Table

Table 2 lists the empirical activation coverage across all 30 layers when retaining $k' = 48$ experts.

Table 2: Per-layer empirical activation coverage (top-48 of 128 experts).

Layer	Cov.	Layer	Cov.	Layer	Cov.	Layer	Cov.
0	∼79%	8	∼80%	16	∼80%	24	∼80%
1	∼79%	9	∼80%	17	∼80%	25	∼80%
2	∼79%	10	∼80%	18	∼80%	26	∼80%
3	∼80%	11	∼80%	19	∼80%	27	∼80%
4	∼80%	12	∼80%	20	∼80%	28	∼80%
5	∼80%	13	∼80%	21	∼80%	29	∼80%
6	∼80%	14	∼80%	22	∼80%		
7	∼80%	15	∼80%	23	∼80%		
Mean: 79.88%							

C Qualitative Verification: Actual Verbatim Model Outputs

To verify that the 48-expert pruned container retains genuine deductive reasoning and does not produce degenerate hallucinations, we report raw, untruncated model generations executed live on the RTX 4050 Laptop GPU at 44.6 TPS.

C.1 Task T4: Syllogistic / Counterfactual Set Logic

Prompt:

```
<|turn>user
If all bloops are razzies and all razzies are lazzies, are all bloops definitely lazzies?
    Explain step by step.
<turn|>
```

```
<|turn>model
```

Verbatim Model Output (Generated at 44.6 Tokens/Sec):

```
[Start thinking]
* Premise 1: "All bloops are razzies." (Bloops \subseteq Razzies)
* Premise 2: "All razzies are lazzies." (Razzies \subseteq Lazzies)
* Question: "Are all bloops definitely lazzies?"
  - This is a classic syllogism/categorical logic problem.
  - It follows the structure of a hypothetical syllogism:
    * Major Premise: All A are B.
    * Minor Premise: All B are C.
    * Conclusion: Therefore, all A are C.

  - Step 1: Identify the first relationship.
    The statement "All bloops are razzies" establishes a group.
    Every member of the set "bloops" belongs to the set "razzies."
    * Relationship: Bloops -> Razzies.

  - Step 2: Identify the second relationship.
    The statement "All razzies are lazzies" establishes a further connection.
    Every member of the set "razzies" belongs to the set "lazzies."
    * Relationship: Razzies -> Lazzies.

  - Step 3: Connect the two relationships (Transitive Property).
    If we follow the chain: Bloops -> Razzies -> Lazzies.
    Since the first group is contained within the second, and the second group
    is contained within the third, the first group must be contained within the third.

  - Step 4: Formulate the conclusion.
    Because if a "bloop" is a "razzi," and being a "razzi" means being a
    "lazzie," then yes, all bloops are definitely lazzies.
```

C.2 Task T1: Python Fibonacci Algorithmic Synthesis

Prompt:

```
<|turn>user
Write a Python function def fibonacci(n): that returns a list of the first n Fibonacci
    numbers.
<turn|>
<|turn>model
```

Verbatim Model Output (Generated at 45.1 Tokens/Sec):

```python
def fibonacci(n):
    if n <= 0:
        return []
    elif n == 1:
        return [0]

    seq = [0, 1]
    while len(seq) < n:
        seq.append(seq[-1] + seq[-2])
    return seq[:n]
```

Notice that boundary conditions ($n \leq 0, n = 1$), list indexing, loop termination, and exact keyword requirements (`def fibonacci`, `return`, `append`, `[0, 1]`) are flawlessly preserved.